TP2 : un module de réservation de salles avec Copilot

Durée : 65 min **Partie** : P2 (complétion, tests, débogage et refactoring)

Objectif

Compléter un module, faire générer des tests depuis ses exigences, corriger un bug, refactoriser et importer des réservations CSV. Relire chaque proposition avant de la garder.

Prérequis

- Démo 2 et TP1 ; environnement commun préparé avec `uv sync`.
- Ouvrir `tps/tp2_module_reservations/` seul dans VS Code.
- Ne modifier ni `test_acceptation_tp2.py` ni `test_import_csv.py`.

Partie 1 — Écrire avec les complétions (15 min)

Dans `reservations.py`, faire compléter `chevauche`, `ajouter` et `taux_occupation` à partir des docstrings. Garder les tests ouverts comme contexte. Si une proposition est fausse, préciser la règle ou utiliser l'inline chat.

```
uv run pytest -q test_acceptation_tp2.py -k "chevauche or ajouter or taux"
```

Pour représenter deux créneaux qui se touchent, utiliser ce contexte dans votre demande :

```
jour = date(2026, 10, 12)
a = Reservation("Turing", datetime(2026, 10, 12, 9),
                datetime(2026, 10, 12, 10), "Alice")
b = Reservation("Turing", datetime(2026, 10, 12, 10),
                datetime(2026, 10, 12, 11), "Bruno")
# Selon le contrat, chevauche(a, b) doit renvoyer False.
```

Partie 2 — Générer des tests depuis les exigences (10 min)

Demander à Copilot des tests dans `test_reservations.py`, à partir des docstrings. Exiger au moins huit cas, dont un planning vide, deux créneaux qui se touchent et un horaire invalide. Relire les valeurs attendues avant de lancer les tests : elles doivent venir des règles métier.

Exemple de test à suivre :

```
def test_creneau_qui_depasse_la_fermeture():
    reservation = Reservation("Turing", datetime(2026, 10, 12, 17),
                              datetime(2026, 10, 12, 19), "Alice")
    with pytest.raises(ValueError):
        ajouter([], reservation)
```

Partie 3 — Corriger le bug des créneaux libres (10 min)

Lancer `uv run pytest -q test_acceptation_tp2.py`. Donner à Copilot le test `test_creneaux_libres_fin_de_journee`, la docstring et le symptôme. Faire corriger l'implémentation, puis expliquer en une phrase pourquoi le dernier créneau manquait. Ne pas modifier le test.

Partie 4 — Refactoriser sous filet de tests (15 min)

Vérifier que `test_rapport_hebdomadaire_inchange` passe. Faire proposer un plan pour simplifier `rapport_hebdomadaire`, puis faire appliquer ce plan : noms explicites, constantes, calcul commun avec `taux_occupation`. Relire le diff et lancer `uv run pytest -q test_acceptation_tp2.py test_reservations.py`. Le résultat du rapport doit rester identique.

Partie 5 — Écrire les règles du projet (5 min)

Faire créer `AGENTS.md` avec les sections Projet, Commandes, Conventions et Tests. Vérifier chaque commande. Ajouter une règle propre au projet avec sa raison, un exemple de test à suivre et l'interdiction de modifier les tests fournis.

Partie 6 — Importer les réservations CSV (10 min)

Faire compléter `charger_csv` en réutilisant `ajouter` pour valider chaque réservation. Le bloc suivant peut servir de point de départ : le faire relire et expliquer par Copilot.

```
import csv

def charger_csv(chemin):
    planning = []
    with open(chemin, encoding="utf-8", newline="") as fichier:
        for ligne in csv.DictReader(fichier, delimiter=";"):
            reservation = Reservation(
                ligne["salle"], datetime.fromisoformat(ligne["debut"]),
                datetime.fromisoformat(ligne["fin"]), ligne["formateur"])
            planning = ajouter(planning, reservation)
    return planning
```

Lancer `uv run pytest -q`. Vérifier que le CSV normal est trié et que `reservations_conflit.csv` provoque bien `ConflitDeReservation`.

Critères de réussite

- Les 19 tests fournis passent : 17 sur le module, 2 sur l'import CSV.
- Vos huit tests ou plus passent ; leurs valeurs attendues viennent des exigences.
- Le rapport reste inchangé après refactoring, et `AGENTS.md` contient des commandes vérifiées.
- Vous pouvez expliquer une correction et sa vérification.

Corrigé

`corrige/reservations.py`, `corrige/test_reservations.py`, `corrige/AGENTS.md` et `corrige/prompts_types.md`.